第二章作业

1

给定一个带头结点的单链表 L，设计一个算法将链表中的所有元素原地逆置，要求只使用 $O(1)$ 的额外空间。

```
void reverseList(ListNode* head) {
    ListNode* pre = nullptr;
    ListNode* next = nullptr;
    ListNode* cur = head->next;
    while (cur != nullptr) {
      next = cur->next;
      cur->next = pre;
      pre = cur;
      cur = next;
    }
    head->next = pre;
  }
```

2

给定一个不带头结点的单链表 L，其中 head 指向第一个数据结点，设计算法找到其中间节点。若节点数为偶数，规定返回两个中间节点中的后一个。要求只遍历链表一次。

```
ListNode* findMidle(ListNode* head) {
  ListNode* slow = head;
  ListNode* fast = head;
  while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
  }
  return slow;
}
```

3

给定两个无环单链表 L_1, L_2，请设计算法判断它们是否相交；若相交，找出它们的第一个公共节点。要求 $O(1)$ 额外空间。

```
ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
    int lenA = 0;
    int lenB = 0;
    ListNode* cur = headA;
    while (cur != nullptr) {
      lenA += 1;
      cur = cur->next;
    }
    cur = headB;
    while (cur != nullptr) {
      lenB += 1;
      cur = cur->next;
    }
```

```
    ListNode* curA = headA;
    ListNode* curB = headB;
    if (lenA > lenB) {
        int cnt = lenA - lenB;
        while (cnt--) {
            curA = curA->next;
        }
    } else {
        int cnt = lenB - lenA;
        while (cnt--) {
            curB = curB->next;
        }
    }
    while (curA != curB) {
        curA = curA->next;
        curB = curB->next;
    }
    return curA;
}
```